

Assignment A1

Winter 2008

Fixed-size bitset

- To be registered on the course, you must show that you are active, and that is done by handing in A1 before the deadline.
- Download this [file](#) and unpack it, eg using the commands **bunzip2 bitset-A1.tar.bz2** and **tar xvf bitset-A1.tar**.
- Your task is to implement a set data type in which the members are integers in the range 0 to 1023.
- Whether an integer is member of a set or not should be represented by only one bit, to save memory.
- The macro **CHAR_BIT** says how many bits there are in a **char**. This is at least eight, but write your code without assuming it is exactly eight.
- **Optional:** If you wish, you may make your code more efficient by using a larger data type than **unsigned char** in order to process more bits per executed machine instruction.
- Implement the empty functions, and type **make**.
 - **void clear_bitset(bitset_t* a)** should remove all members from the bitset, ie set every bit to zero (or false).
 - **void set_bit(bitset_t* a, unsigned int pos)** set one bit (at position **pos**) to true.
 - **void clear_bit(bitset_t* a, unsigned int pos)** set one bit (at position **pos**) to false.
 - **void print_bitset(bitset_t* a)** print out which integers are members of the set.
 - **void bitset_union(bitset_t* a, bitset_t* b, bitset_t* c)** Set union, ie **c = a union b**.
 - **void bitset_intersection(bitset_t* a, bitset_t* b, bitset_t* c)** Set intersection, ie **c = a intersection b**.
- Type **make test** to check that your program produces the correct output. If it does not, the program **diff** prints out the differences between your output and the correct output (stored in the file **correct**).

Most recently modified by *js* at *Mon Dec 17 14:02:38 CET 2007*